

Oracle Cloud VM — Aave V3 Liquidator Operator Guide

Audience: Someone who may never have used Linux before.

Goal: Get the bot running safely on your Oracle Virtual Machine.

Date: July 2026

Recommended first mode: Shadow soak (watches the market; does **not** spend money)

Read this first (safety)

1. **Start with soak mode only.** Soak mode watches prices and positions but does **not** send live liquidation transactions.
2. **Do not start “production / live” mode** until your engineer confirms all of these are done:
 - A successful ~48 hour soak with healthy logs
 - A **v5** LiquidationFlashReceiver contract deployed on Base
 - Wallet funded and Sepolia/mainnet gates your team requires
3. **Never share** your private key, RPC API keys, or `.env` files with anyone (including screenshots in chat).
4. If anything looks wrong, stop the bot (Step 10) and contact your engineer before continuing.

What you need before you begin

Item	Why
Oracle Cloud VM (Ubuntu Linux)	Where the bot runs 24/7
SSH access (or Oracle Cloud Console “Cloud Shell / Terminal”)	So you can type commands
GitHub access to this repo	To download / update the code
RPC URLs (HTTP + WebSocket) for Base	Bot talks to the blockchain
Wallet private key (only for later live mode)	Signs transactions — not needed for soak
About 30–60 minutes for first-time setup	Plus waiting time for soak

Part A — Open a terminal on the VM

Option 1 — Oracle Cloud Console (easiest)

1. Open a web browser.
2. Go to <https://cloud.oracle.com> and sign in.
3. Open **Compute** → **Instances**.
4. Click your bot VM name.
5. Click **Console connection** or use **Cloud Shell** / the instance terminal button your tenancy provides.
6. Wait until you see a black or dark screen with a blinking cursor. That is your **terminal**.

Option 2 — SSH from your Windows PC (optional)

1. Open **PowerShell** on your PC.
2. Type a command like this (replace with your real IP and key path):

```
ssh -i "C:\path\to\your-ssh-key.key" ubuntu@YOUR.VM.PUBLIC.IP
```

3. Press Enter. If asked “Are you sure...?”, type `yes` and press Enter.
4. You should see a Linux prompt such as `ubuntu@bot-vm:~$`.

Part B — One-time setup (do this only the first time)

Copy **one line at a time**. Press **Enter** after each line. Wait until the command finishes before starting the next.

Step 1 — Go to your home folder

```
cd ~
```

Step 2 — Install basic tools (if not already installed)

```
sudo apt-get update
sudo apt-get install -y git curl build-essential
```

If asked to confirm, type `y` and press Enter.

Step 3 — Install Node.js 20 (if node -v fails or is too old)

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs
node -v
npm -v
```

You want Node **v20** or newer. If `node -v` shows something like `v20.x.x`, you are good.

Step 4 — Install PM2 and Redis tools

```
sudo npm i -g pm2
sudo apt-get install -y redis-server
sudo systemctl enable redis-server
sudo systemctl start redis-server
```

Step 5 — Download the bot code

If the folder does **not** exist yet:

```
cd ~
git clone https://github.com/BlancoChukwu/Bot2.1.git optimism-aave-v3-liquidator-ts
cd ~/optimism-aave-v3-liquidator-ts
```

If the folder **already** exists:

```
cd ~/optimism-aave-v3-liquidator-ts
```

Step 6 — Switch to the correct branch and update

```
git fetch origin
git checkout feat/receiver-v5-oracle-floor-quote-estimator
git pull --ff-only
```

If `git pull` prints an error about conflicts, **stop** and message your engineer. Do not guess.

Step 7 — Install project dependencies

```
cd ~/optimism-aave-v3-liquidator-ts
npm install
```

This can take several minutes. Wait until you see the prompt again.

Step 8 — Create / refresh environment profile files

```
npm run env:bootstrap
npm run env:merge-example -- --target .env.event-purity-soak
npm run env:merge-example -- --target .env.event-purity-production
npm run env:sync-receiver-v5
```

What this does (in plain English):

- Builds the soak and production settings files
- Adds any new required settings from the template
- Sets receiver version to **5**
- Removes the old global swap-fee override so the safer per-pair fee map is used

Step 9 — Put your secrets into the soak file

Open the soak settings file in a simple editor:

```
nano .env.event-purity-soak
```

How to use nano (no computer skills needed):

1. Use arrow keys to move around.
2. Find these lines and fill them with values from your engineer (do not invent them):

Setting	What to put
RPC_URL= or EXECUTION_RPC_URL_PRIMARY=	Your Base HTTPS RPC URL
WS_RPC_URL_PRIMARY=	Your Base WebSocket URL (must start with <code>wss://</code>)
CHAIN=base	Leave as <code>base</code>
SIMULATION_MODE=true	Must stay <code>true</code> for soak
ENABLE_LIVE_TX=false	Must stay <code>false</code> for soak
LIQUIDATION_RECEIVER_EXPECTED_VERSION=5	Must be <code>5</code>
LIQUIDATION_SWAP_SLIPPAGE_BPS=200	Must be <code>200</code> unless engineer says otherwise

3. Confirm these are correct for soak:

- `ENABLE_LIVE_TX=false`
- `SIMULATION_MODE=true`
- There should be **no** active line `LIQUIDATION_SWAP_POOL_FEE=...` (if you see one, delete that whole line)

4. Save and exit nano:

- Press `Ctrl + O` (letter O), then Enter to save
- Press `Ctrl + X` to exit

Step 10 — Make starter scripts runnable

```
chmod +x start-event-purity-soak.sh start-event-purity-production.sh watch-bot.sh scripts/launche
```

Part C — Start the bot in safe soak mode (do this every time you want soak)

Step 11 — Start soak

```
cd ~/optimism-aave-v3-liquidator-ts  
./start-event-purity-soak.sh
```

What you should see:

- Messages about stopping any old bot
- Redis check
- Build completing
- Preflight OK
- Bot launching in the background

If the script stops with red errors, copy the last 30 lines and send them to your engineer. Do not start production.

Step 12 — Confirm it is running

```
npm run bot:status  
./watch-bot.sh --once
```

Healthy signs:

- Status shows a running process
- Health endpoint works (optional check below)
- Log file is growing under `logs/`

Optional health check:

```
curl -s http://127.0.0.1:9090/healthz
```

You want a response that looks successful (not a connection error).

Step 13 — Watch the live monitor (recommended)

```
./watch-bot.sh
```

- The screen refreshes about every 30 seconds.
- To stop watching (bot keeps running): press `Ctrl + C`.

Step 14 — Follow the log file

```
ls -lt logs/event-purity-soak-*.log | head
```

Then open the newest file (replace the filename with the one listed):

```
tail -f logs/event-purity-soak-YYYYMMDD-HHMMSS.log
```

Press `Ctrl + C` to stop following the log (bot keeps running).

Part D — What “good” looks like during soak

Leave soak running for about **48 hours** unless your engineer says otherwise.

Good signs in logs / monitor:

- Bot stays up (no constant restarts)
- Prices refresh without long outages
- No repeated fatal errors
- Memory stays below warning levels your engineer defined

Bad signs — **stop and ask for help**:

- Immediate crash loop (starts, dies, starts, dies)
- Missing RPC / WebSocket errors that never recover
- Messages about invalid oracle feeds that do not clear
- You accidentally set `ENABLE_LIVE_TX=true` during soak

To stop the bot safely:

```
cd ~/optimism-aave-v3-liquidator-ts
npm run bot:stop
```

Part E — Day-to-day commands (cheat sheet)

Always start in the project folder:

```
cd ~/optimism-aave-v3-liquidator-ts
```

What you want	Command
Start safe soak	<code>./start-event-purity-soak.sh</code>
See if bot is running	<code>npm run bot:status</code>
Live dashboard	<code>./watch-bot.sh</code>
One status snapshot	<code>./watch-bot.sh --once</code>
Stop bot	<code>npm run bot:stop</code>
Update code then restart soak	<code>git pull --ff-only</code> then <code>./start-event-purity-soak.sh</code>
Sync new settings keys	<code>npm run env:merge-example -- --target .env.event-purity-soak</code> then <code>npm run env:sync-receiver-v5</code>

After updates, prefer:

```
git pull --ff-only
npm install
```

```
npm run env:sync-receiver-v5
./start-event-purity-soak.sh
```

Part F — Live / production mode (ONLY after engineer approval)

Do not do this section until explicitly told. Live mode can spend gas and attempt real liquidations.

Prerequisites checklist (all must be true)

- ☐ ~48h soak completed cleanly
- ☐ Engineer confirms HF skip-storm / soak review passed
- ☐ v5 LiquidationFlashReceiver deployed on Base
- ☐ `.env.event-purity-production` has the **new** receiver address
- ☐ `LIQUIDATION_RECEIVER_EXPECTED_VERSION=5`
- ☐ `LIQUIDATION_AUTHORIZED_INITIATOR` equals your bot wallet address
- ☐ Wallet has enough ETH/Base gas funds
- ☐ `ENABLE_LIVE_TX=true` only in the **production** env file (not soak)

Live start sequence

1. Edit production env:

```
nano .env.event-purity-production
```

Confirm:

- `ENABLE_LIVE_TX=true`
- `SIMULATION_MODE=false`
- `LIQUIDATION_RECEIVER_EXPECTED_VERSION=5`
- Receiver address is the **v5** contract
- No active `LIQUIDATION_SWAP_POOL_FEE=` line

2. Sync keys:

```
npm run env:merge-example -- --target .env.event-purity-production
npm run env:sync-receiver-v5
```

3. Create dry-run receipt (required; expires in ~15 minutes):

```
npm run dry-run:receipt:event-purity-production
```

4. Within **15 minutes**, start live:

```
./start-event-purity-production.sh
```

5. Monitor:

```
./watch-bot.sh
```

6. Emergency stop:

```
npm run bot:stop
```

Part G — Common problems and simple fixes

“Permission denied” when starting ./start-...sh

```
chmod +x start-event-purity-soak.sh start-event-purity-production.sh watch-bot.sh scripts/launch
```

“Missing .env.event-purity-soak”

```
npm run env:bootstrap
npm run env:merge-example -- --target .env.event-purity-soak
npm run env:sync-receiver-v5
```

Then fill RPC/WS values in nano again.

“pm2 not found”

```
sudo npm i -g pm2
```

Redis / ensure-redis fails

```
sudo systemctl start redis-server
sudo systemctl status redis-server
```

Preflight complains about WebSocket

- Open `.env.event-purity-soak`
- Make sure `WS_RPC_URL_PRIMARY=` is a real `wss://...` URL
- Make sure `FLASHBLOCKS_ENABLED=true`

Warning: LIQUIDATION_SWAP_POOL_FEE is set

Delete that line from the env file, save, then:

```
npm run env:sync-receiver-v5
./start-event-purity-soak.sh
```

Bot was running, then machine rebooted

```
cd ~/optimism-aave-v3-liquidator-ts
sudo systemctl start redis-server
./start-event-purity-soak.sh
```

Part H — Settings that must match the latest code

These should already be set after `npm run env:sync-receiver-v5`:

Key	Soak	Live
ENABLE_LIVE_TX	false	true (only when approved)
SIMULATION_MODE	true	false
LIQUIDATION_RECEIVER_EXPECTED_VERSION	5	5
LIQUIDATION_SWAP_SLIPPAGE_BPS	200	200
LIQUIDATION_SWAP_POOL_FEE	unset	unset
USE_EVENT_WATCHLIST	true	true
USE_PIPELINE_ORCHESTRATOR	true	true
FLASHBLOCKS_ENABLED	true	true

Built-in (no env needed): Uniswap TWAP oracle sanity, thin-pair live TVL probe, in-flight drain (~60s), PM2 kill timeout (75s), recent-attempt ledger under `.runtime/`.

Part I — Who to call / what to send when stuck

Send your engineer:

1. Exact command you ran
2. Last 40 lines of the terminal output
3. Output of:

```
cd ~/optimism-aave-v3-liquidator-ts
git rev-parse --short HEAD
git branch --show-current
npm run bot:status
./watch-bot.sh --once
```

Never paste private keys or full `.env` contents.

Quick start card (after first-time setup is done)

```
cd ~/optimism-aave-v3-liquidator-ts
git pull --ff-only
npm run env:sync-receiver-v5
./start-event-purity-soak.sh
./watch-bot.sh
```

To stop:

```
npm run bot:stop
```

End of guide. Prefer soak mode until your engineer says live is cleared.